What should be unique?

The relationship between the `Venue` and `Room` objects is based on their ID values, which are automatically generated by the database. Two different venues could both have rooms named `Red` without a problem, as the ID is what determines the relationship, not the value of the `name` field.

In fact, there is nothing currently stopping you from creating two `Room` objects with the same name and pointing them both at the same `Venue`. Django provides mechanisms for controlling this, which are discussed later in this chapter.

In your best Ron Popeil voice, read the next sentence out loud. *But wait, there's more!* In addition to the forward relationship being accessible, you can go backward as well. The object that a `ForeignKey` field points to automatically gets a query manager to access the associated objects. The query manager is named after the associated model with `_set` stuck on the end. Your `Venue` model has a `.room_set` query manager. All the queries you learned to use on the `.objects` query manager apply to this one, with the difference being the objects queried are only those associated through a foreign key. Run the `.all()` query on the `Venue` instance's `.room_set`:

```
>>> cbgb.room_set.all()
<QuerySet [<Room: Room(id=1, name=Red)>, <Room: Room(id=2, name=Blue)>]>
```

Because `.room_set` is an attribute and queries are callables, you can pass a `Venue` object into a template and iterate over the associated rooms using `{% for room in venue .room_set.all %}`. In fact, one of the exercises in this chapter asks you to do just that.

The related name

You can control what Django calls an associated model by adding the `related_name` parameter to the field. For example, to use `my_rooms` instead of `room_set` in the previous example, you would add `related_name="my_rooms"` to the `venue` ForeignKey field on the `Room` class. You can also change how the name appears in a query using the `related_query_name` field in a similar manner.

Generally, you shouldn't rename this relationship without a good reason; you'll only cause confusion for the people maintaining your code. There are two cases where this comes in handy. One, if your model includes two `ForeignKey` relationships to the same model, one needs to be renamed. The other is a special case: by setting `related_name="+"`, the reverse relationship is turned off, which you may wish to do for efficiency. For details, see the documentation: http://mng.bz/oe5D.

In section 4.4.2 you learned about field look-ups. A similar mechanism using double underscores (__) can also be used to query based on fields and relationships. For example, you can query for a `Venue` by filtering on `room__name="Red"`. This finds any `Venue` objects that have an associated `Room` object with `Red` as their `name` attribute.